

SOC/L1 - 8.4. Detecting Web DDoS

Task 1 - Introduction

Denial-of-Service (DoS) attacks can take many forms, but their ultimate aim is to disrupt or completely block access to a website or web service.

Task 2 - DoS and DDoS Attacks

DoS attacks can be launched against different layers of a system. However, in this room, we'll focus on the application layer (Layer 7) of the OSI model, where websites and web applications are often targeted.

Denial-of-Service (DoS)

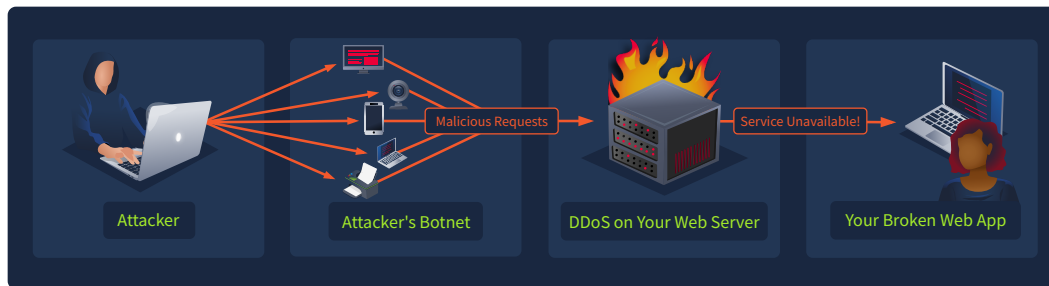


DoS attack is a malicious attempt to make a website, server, or network resource unavailable to legitimate users. Instead of stealing data or breaking into a account, the attacker simply tries to crash the target or tie up its resources so real users get locked out. How It Happens:

- **Overwhelming Resources (Volume-based):** Flooding a server with more traffic or requests than its CPU, RAM, or bandwidth can process.
- **Exploiting Vulnerabilities (Application-based):** Sending specially crafted, malicious input (like corrupt data into a search bar) that triggers a bug, causing the application to hang or freeze.

Note on DDoS: A standard **DoS** attack originates from a single computer or IP address. A **DDoS** (Distributed Denial-of-Service) uses an entire network of infected computers (a botnet) to attack from millions of locations at once, making it much harder to block.

Distributed Denial-of-Service (DDoS)



The limitation of a basic DoS attack is that it relies on a single machine and a single internet connection. While one computer can generate many requests, its impact is capped by its CPU, memory, bandwidth, and network.

To scale up, attackers turn to **Distributed Denial-of-Service (DDoS)** attacks and utilize botnets, an army of compromised devices under their control. The computers, IoT devices, and even servers are often infected with malware and secretly controlled by the attacker. When instructed, the bots flood the target website or web application with traffic, overwhelming it much more effectively than a single machine could.

Types of Denial-of-Service Attacks

These attacks can be carried out by a single attacker (**DoS**) or distributed across a botnet (**DDoS**).

DoS Attack Type	Description
Slowloris	Sending many partial HTTP requests to tie up server resources
HTTP Flood	Sending a large number of HTTP requests to overwhelm the server
Cache Bypass	Bypassing CDN edge servers and forcing the origin server to respond
Oversized Query	Forcing the server to process large, resource-intensive requests

DoS Attack Type	Description
Login/Form Abuse	Overloading authentication logic with login attempts or password resets
Faulty Input Validation Abuse	Exploiting poorly designed input handling

Answer the questions below

What class of attack relies on disrupting the availability of a web service?

Denial-of-Service

What do we call the network of compromised machines that attackers use to launch DDoS attacks?

Botnet

Task 3 - Attack Motives

Possible Attack Motives

Motive	Description	Example Scenario
Financial Loss(opens in new tab)	Disrupt services to stop or reduce sales and revenue	Flooding an e-commerce website during peak holiday sales
Extortion(opens in new tab)	Demand payment to stop a current attack	Threatening a bank with a ransom DDoS
Hacktivism(opens in new tab)	Disruption for social or political protest	Attacking government websites during election season
Distraction(opens in new tab)	Redirect defenders' attention while other attacks take place	Launching a DDoS while attacking other infrastructure
Competition(opens in new tab)	Disrupt a rival's service to drive up their costs or gain market share	A competitor launches a DDoS during a product launch
Denial of Wallet(opens in new tab)	Force the victim to rack up service usage costs	Attackers repeatedly access AWS S3 data, generating costs per request

Motive	Description	Example Scenario
Reputational Damage (opens in new tab).	Cause customers to lose trust in a company	Game servers crashing during launch day

In the Wild

On New Year's Eve 2015, the [BBC](#)(opens in new tab) was the victim of a DDoS attack that took its website offline. The site's readers could not access articles for several hours as their requests timed out or returned internal error messages. The group that claimed responsibility, [New World Hacking](#)(opens in new tab), says it carried out the DDoS simply as a test of its capabilities.

In 2023, [Microsoft](#)(opens in new tab) experienced a large-scale layer 7 DDoS attack that caused Azure, OneDrive, and Outlook outages. The hacktivist group [Anonymous Sudan](#)(opens in new tab) claimed responsibility. The attackers used techniques such as HTTP flooding and Slowloris to overwhelm web servers, causing major disruptions for Microsoft customers around the world.

Answer the questions below

Which attacker motive aims to make customers lose confidence in a company?

Reputational Damage

Which motive most likely drove the 2023 DDoS attack against Microsoft?

Hacktivism

Task 4 - Log Analysis

Web server logs are a valuable source of evidence when investigating denial-of-service attacks. Every major web service, whether Apache, NGINX, or Microsoft IIS, records web requests in a somewhat standardized log format. By examining these logs, analysts and responders can uncover patterns that help distinguish between normal user traffic and malicious activity.

Take a look at the indicators below:

Indicator	Example	Description
High Request Rate	<code>10.10.10.100</code> → 1000 <code>GET /login</code>	A resource-heavy page like <code>/login</code> is flooded with requests to overwhelm authentication processes. Login pages are common targets since each request may trigger password checks and database queries
Odd User-Agents	<code>curl/7.6.88</code> → <code>/index</code> repeatedly	Attackers spoof outdated or unusual User-Agents to blend in or bypass filters. Spotting traffic with tools like <code>curl</code> or <code>Python-urllib/3.x</code> , for example, can be a red flag for automated attacks
Geographic Anomalies	IP address origins dotted around the world	Legitimate traffic typically comes from a few regions where real users are located. A globally distributed botnet may utilize IP addresses from around the world
Burst Timestamps	50 requests in 1 second → <code>/search</code>	A sudden spike of requests packed into the same second creates an unnatural traffic pattern that points to automation
Server Errors (5xx(opens in new tab))	A significant spike of <code>503 Service Unavailable</code> errors	A sudden surge of server error responses (<code>500</code> - <code>511</code>) indicates resources are maxed out and the service is struggling under attack traffic
Logic Abuse	<code>GET /products?limit=999999</code>	Attackers craft queries that overload the server, forcing it to load huge amounts of information and slowing it down for everyone

Targeted Resources

Commonly targeted endpoints and reasoning:

- `/login` - involves authentication processes
- `/search` - requires complex database queries
- `/api` endpoints - critical for dynamic content delivery
- `/register` or `/signup` - requires database writes and validation
- `/contact` or `/feedback` - requires database entries and can trigger email notifications
- `/cart` or `/checkout` - requires session management, inventory checks, and payment processing

Log Sample

1. **Normal User Traffic** - Every few seconds, a user requests a page and receives a response as expected
2. **DoS Attack** - Beginning at `10:01:10`, you can see the IP address `203.0.113.55` begin to send repeated `GET` requests to `/login.php`
3. **Web Server Down** - Users are requesting pages and receiving `503` responses indicating the service is unavailable

```

192.168.1.10 [31/Aug/2025:10:01:02] "GET /index.html HTTP/1.1" 200 4523 "Mozilla/5.0"
192.168.1.15 [31/Aug/2025:10:01:05] "GET /products.html HTTP/1.1" 200 6112 "Mozilla/5.0"
192.168.1.20 [31/Aug/2025:10:01:09] "GET /contact.html HTTP/1.1" 200 2987 "Mozilla/5.0"

203.0.113.55 [31/Aug/2025:10:01:10] "GET /login.php HTTP/1.1" 200 245 "curl/7.68.0"
203.0.113.55 [31/Aug/2025:10:01:10] "GET /login.php HTTP/1.1" 200 245 "curl/7.68.0"
203.0.113.55 [31/Aug/2025:10:01:10] "GET /login.php HTTP/1.1" 200 245 "curl/7.68.0"
203.0.113.55 [31/Aug/2025:10:01:10] "GET /login.php HTTP/1.1" 200 245 "curl/7.68.0"
203.0.113.55 [31/Aug/2025:10:01:10] "GET /login.php HTTP/1.1" 200 245 "curl/7.68.0"
...100 more lines

192.168.1.30 [31/Aug/2025:10:01:20] "GET /index.html HTTP/1.1" 503 0 "Mozilla/5.0"
192.168.1.33 [31/Aug/2025:10:01:21] "GET /products.html HTTP/1.1" 503 0 "Mozilla/5.0"
192.168.1.33 [31/Aug/2025:10:01:26] "GET /products.html HTTP/1.1" 503 0 "Mozilla/5.0"

```

Hands On

Your bicycle parts website has undergone a denial-of-service attack. Open up the `access.log` file on the user's Desktop to begin your investigation. The logs include a mix of normal user-generated traffic and attacker traffic. While you

comb the logs, be on the lookout for repeated requests to the same page and remember the indicators you learned about in this task. Best of luck!



Answer the questions below

What is the attacker's IP address?

203.12.23.195

vim access.log (The attack started with a curl command.)

```
192.168.1.10 - - [31/Aug/2025:01:58:23 +0000] "GET /products/camera HTTP/1.1" 304 246 "-" "Mozilla/5.0 (Linux x86_64; rv:128.0)
192.168.1.5 - - [31/Aug/2025:01:58:29 +0000] "GET /support/help HTTP/1.1" 200 256 "-" "Mozilla/5.0 (Windows 10.0; Win64; x64)
203.12.23.195 - - [31/Aug/2025:01:59:08 +0000] "GET /login HTTP/1.1" 200 519 "-" "curl/7.88.1"
203.12.23.195 - - [31/Aug/2025:01:59:08 +0000] "GET /login HTTP/1.1" 200 519 "-" "curl/7.88.1"
203.12.23.195 - - [31/Aug/2025:01:59:08 +0000] "GET /login HTTP/1.1" 200 519 "-" "curl/7.88.1"
203.12.23.195 - - [31/Aug/2025:01:59:08 +0000] "GET /login HTTP/1.1" 200 519 "-" "curl/7.88.1"
203.12.23.195 - - [31/Aug/2025:01:59:08 +0000] "GET /login HTTP/1.1" 200 519 "-" "curl/7.88.1"
203.12.23.195 - - [31/Aug/2025:01:59:08 +0000] "GET /login HTTP/1.1" 200 519 "-" "curl/7.88.1"
```

Which page is repeatedly targeted by the attacker's requests?

/login

```
203.12.23.195 - - [31/Aug/2025:01:59:08 +0000] "GET /login HTTP/1.1" 200 519 "-" "curl/7.88.1"
203.12.23.195 - - [31/Aug/2025:01:59:08 +0000] "GET /login HTTP/1.1" 200 519 "-" "curl/7.88.1"
203.12.23.195 - - [31/Aug/2025:01:59:08 +0000] "GET /login HTTP/1.1" 200 519 "-" "curl/7.88.1"
203.12.23.195 - - [31/Aug/2025:01:59:08 +0000] "GET /login HTTP/1.1" 200 519 "-" "curl/7.88.1"
203.12.23.195 - - [31/Aug/2025:01:59:08 +0000] "GET /login HTTP/1.1" 503 267 "-" "curl/7.88.1"
203.12.23.195 - - [31/Aug/2025:01:59:08 +0000] "GET /login HTTP/1.1" 503 267 "-" "curl/7.88.1"
203.12.23.195 - - [31/Aug/2025:01:59:08 +0000] "GET /login HTTP/1.1" 503 267 "-" "curl/7.88.1"
203.12.23.195 - - [31/Aug/2025:01:59:08 +0000] "GET /login HTTP/1.1" 503 267 "-" "curl/7.88.1"
```

After the attack, what error code do legitimate users receive?

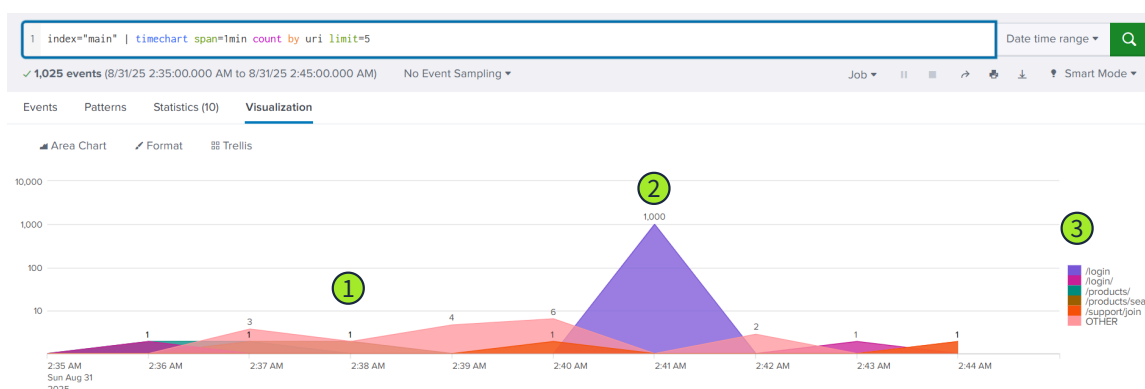
503

Task 5 - Leveraging SIEMs

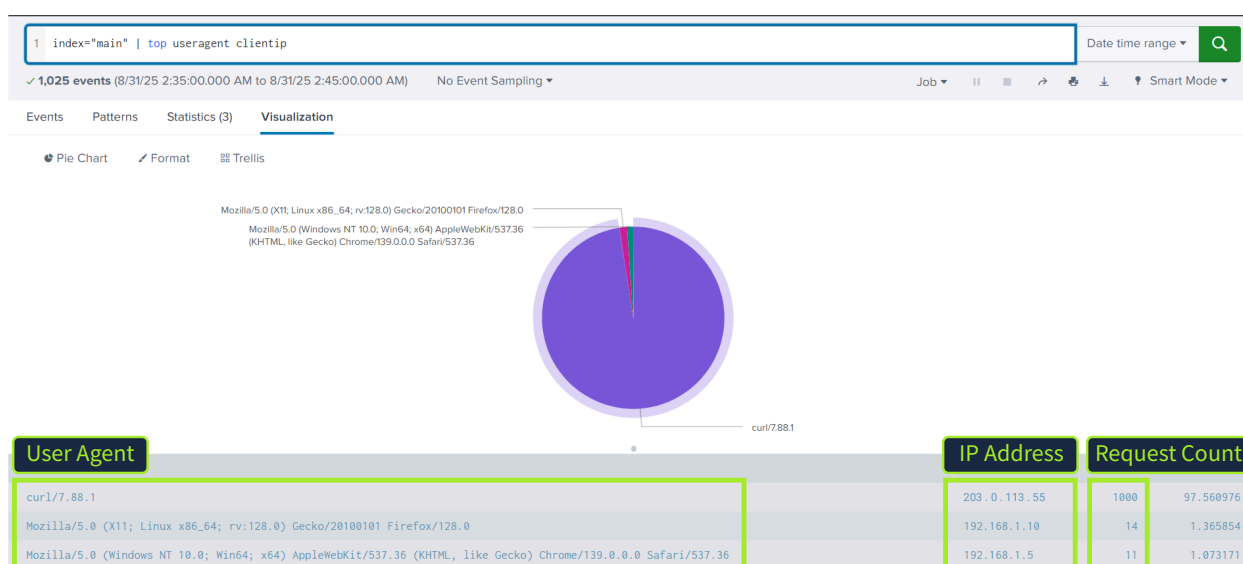
Security Information and Event Management (SIEM) platforms make log analysis far more efficient by providing the ability to combine multiple log sources and

query for useful fields. Instead of scrolling through endless raw entries in a log file, you can filter and sort logs by IP address, user agent, or response code. This ability makes identifying patterns in traffic much easier.

1. **Normal User Requests** A few requests to various pages every minute
2. **DoS Attack** - 1,000 requests to `/login.php` within a one-minute timeframe
3. **Requested Pages**



Looking over the same requests but filtering by the user agent (`useragent`) and IP address (`clientip`) (**IP address of the client device making a request to a server**) fields enables you to see more details about where the requests originated.



Hands On

During this exercise, you'll analyze web access logs collected during the suspected attack period. These logs contain a mix of normal user traffic and potentially malicious requests. To begin, open the **Search & Reporting** app in Splunk and run a search in the main index: `index="main"`. On the left side of the GUI, you will see the fields that have been extracted from the log file. These will prove to be very useful as you investigate.

Answer the questions below

What was the most frequently requested uri?

/search

The screenshot shows the Splunk Search & Reporting app interface. On the left, a list of extracted fields is visible, with 'uri' selected. On the right, the 'uri' field is chosen for analysis, showing '20 Values, 100% of events'. Below this, there are tabs for 'Reports', 'Top values', 'Top values by time', and 'Rare values'. The 'Top values' report is active, displaying a table of the top 10 values for the 'uri' field.

Top 10 Values	Count	%
/search	2,068	81.804%
/products/brakes	35	1.384%
/static/css/style.css	34	1.345%
/product/5678	30	1.187%
/products/tires	30	1.187%
/faq	29	1.147%
/contact	27	1.068%
/login	27	1.068%
/support	27	1.068%
/images/logo.png	26	1.028%

or

```
index=main
| stat count AS requests BY uri
sort - requests
```

Which clientip made the most requests to the target uri?

203.0.113.7

```
index = main uri=/search  
| top clientip
```

How many IP addresses were part of the botnet that attacked your website?

60

```
index = main uri=/search  
| stats dc(clientip) AS botnetip count AS total
```

Which useragent was most commonly used by the attacking traffic?

Java/1.8.0_181

```
index = main uri=/search  
| top useragent
```

Use the timechart command to visualize the requests. What is the peak number of requests made per second during the attack?

207

```
index = main uri=/search  
| timechart span=1s count by uri limit=5
```

Which legitimate (non-attacking) clientip received the first 503 response status post-attack?

10.10.0.27

```
index = main
| table _time clientip useragent uri
| sort -_time
```

Task 6 - Defense

Attackers constantly look for weak points to exploit, but defenders have various tools and methods to keep systems resilient.

1. Application Level Defense

Secure Development Practices: Input validation prevents denial-of-service attacks by enforcing strict rules on user input to keep processing fast and secure.

CAPTCHA: One way to stop automated traffic is to require a challenge before granting access. This could be a CAPTCHA, where the user solves a puzzle, like clicking images or checking a box. For humans, it's a small step, but for bots, it can block or slow down an attack. Websites can also use JavaScript challenges, which run quietly in the background to confirm if a visitor is a real user or automated traffic. Legitimate users usually don't notice them, but automated tools and botnets often fail, making these challenges an effective filter against malicious traffic.

1. How It Works

CAPTCHAs present challenges that are easy for humans to solve but difficult for automated scripts, such as:

- **Text Recognition:** Identifying distorted letters or numbers.
- **Image Selection:** Clicking on specific objects (e.g., crosswalks, traffic lights, cars).
- **Behavioral Analysis (reCAPTCHA v2/v3):** Tracking mouse movements, timing, or device telemetry (often reduced to a simple *"I'm not a robot"* checkbox or completely invisible background scoring).

2. Role in Application Security & DDoS Defense

In web application security and WAF (Web Application Firewall) architectures, CAPTCHA serves as a crucial **rate-limiting and mitigation challenge**:

- **Throttling Botnets:** When a WAF detects a sudden spike in traffic or suspicious request patterns to high-cost endpoints (like `/login.php` or a search form), it forces suspicious IPs to solve a CAPTCHA before processing their request.
- **Preserving Origin Resources:** Automated attack scripts (like volumetric DoS or credential stuffing bots) generally fail or abandon requests when hit with a CAPTCHA, preventing them from hitting the origin server.
- **Differentiating Traffic:** It allows legitimate human traffic to proceed without outright blocking entire IP blocks during an attack mitigation event.

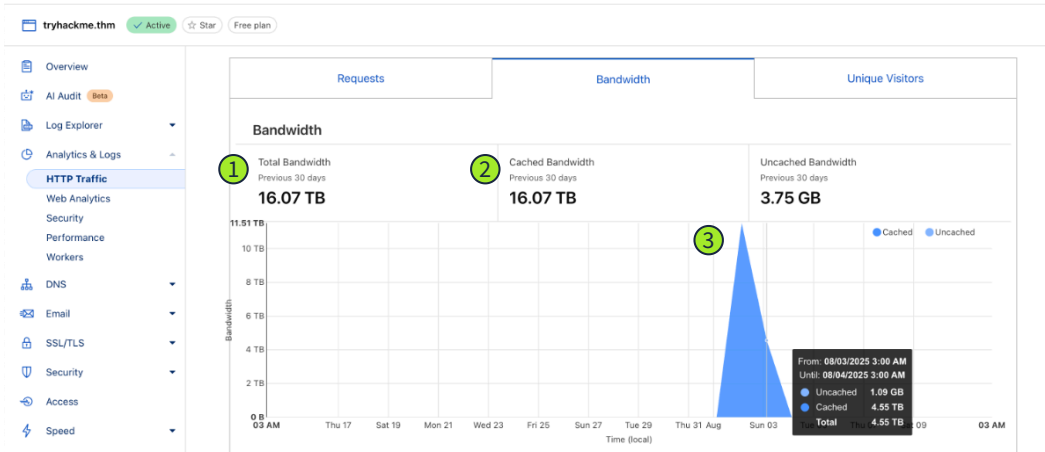


2. Network and Infrastructure Defenses

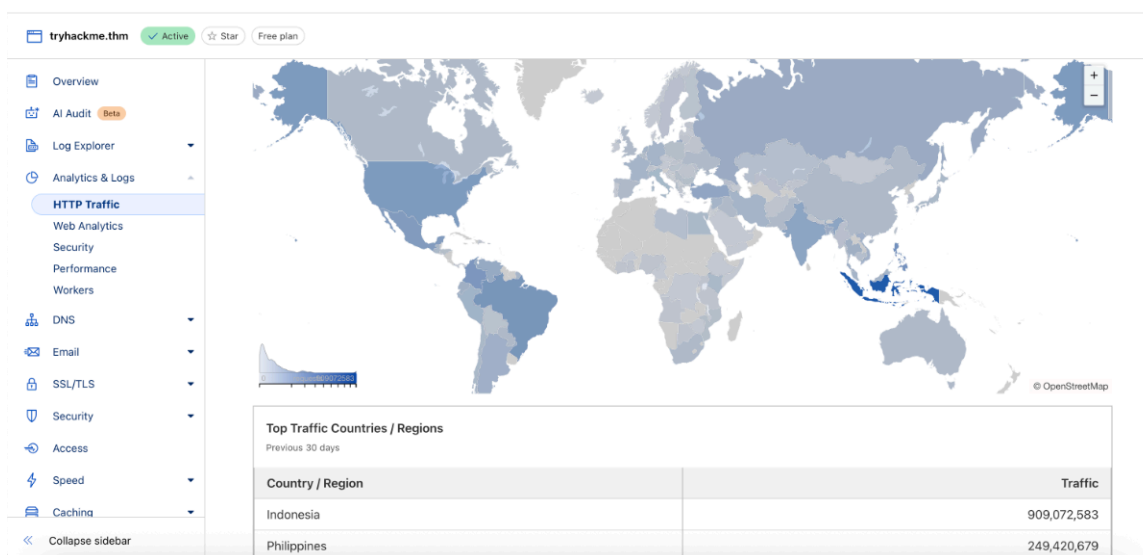
Content Delivery Network (CDN): CDNs reduce server load and mitigate DDoS attacks by caching content on distributed edge servers (**A server geographically positioned close to end-users to process requests, serve cached content locally, and shield the main origin server from traffic spikes.**) and load-balancing traffic across origin servers.

Here's an example of a 16 TB DDoS attack displayed in the Cloudflare CDN dashboard.

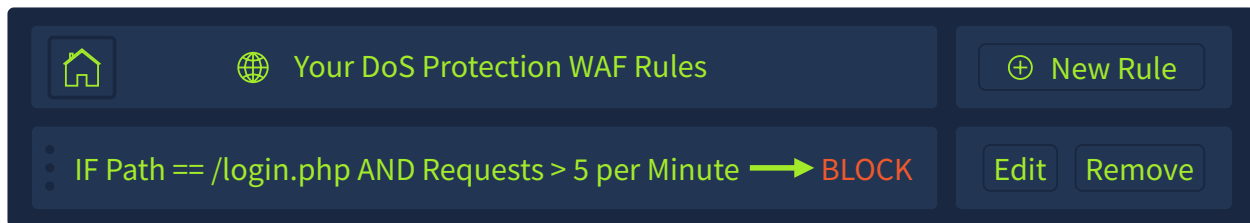
1. **Total bandwidth over a 30-day period** - This shows the entire traffic volume over the last 30 days. If your site normally sees a few hundred gigabytes a month, suddenly seeing 16 TB indicates something is abnormal.
2. **Total cached bandwidth by CDN edge servers** - This represents how much traffic was successfully delivered by the CDN's edge servers. Nearly all of the bandwidth being cached indicates that Cloudflare absorbed the attack traffic before it could overwhelm the backend.
3. **Traffic spike from DDoS attack** - The spike in the graph is the signature of the DDoS attack. Without a CDN, the flood of requests would have hit the origin server.



Beyond absorbing traffic, CDNs also provide analysts with powerful visibility, including helpful visuals and diagnostics of website traffic. This lets you quickly break down requests by geographic location, volume, and source patterns, helping distinguish malicious traffic from legitimate users.



Web Application Firewall (WAF) : WAFs inspect incoming traffic to allow, challenge, or block requests based on predefined rules, known threat intelligence, and custom settings. **Mitigation & Defense:** Integrated within CDNs, modern WAFs protect web applications from DoS/DDoS attacks by enforcing measures like rate-limiting on sensitive endpoints to filter out malicious traffic.



Large-Scale Mitigation: Modern defense solutions can block denial-of-service attacks and also leverage their vast distributed infrastructure to absorb and balance large volumes of traffic. In 2023, [Google\(opens in new tab\)](#) mitigated a large-scale DDoS that peaked at 398 million requests per second. [Cloudflare\(opens in new tab\)](#) claims to have mitigated the largest DDoS ever recorded, which peaked at 11.5 Tbps and lasted only 35 seconds. These examples demonstrate how providers use global networks and traffic filtering to keep even the largest attacks from taking critical services offline.

Bypassing Security Measures: While CDNs, caching, and WAFs provide strong protection, attackers often attempt to bypass these defenses. One technique is appending random query parameters. Your CDN might serve a cached URL at `/products`, but if an attacker appends the query with a random string like `/products?a=abcd`, your CDN cannot serve the cached page, and the origin server is forced to respond. Similarly, changing user agents, spoofing referrer pages, or launching requests from diverse geographic regions can help attackers evade WAF filtering rules.

Answer the questions below

What type of security challenge blocks bots by asking users to solve a simple puzzle?

CAPTCHA (It is a security mechanism used by websites to verify whether an incoming request is being made by a real human or an automated bot/script.)

Which CDN feature spreads traffic across multiple servers to prevent overload?

Load-balancing